

한성 캠퍼스 라이프

대학생을 위한 시간표, 과제, 메모 통합 관리 앱

모바일프로그래밍 기말 프로젝트

AI응용학과 | 강선우

Kotlin | Android Studio | Pixel 3a 에뮬레이터

7:24 3G +					
한성 캠퍼스 라이프					
시간표 과제 메모					
	월	화	수	목	금
09:00	모바일 프로 그래밍 공학관 407 호 09:00~10:00				
10:00					
11:00					
12:00					
13:00					
14:00					
15:00					
16:00					
17:00					

왜 이 주제를 골랐나

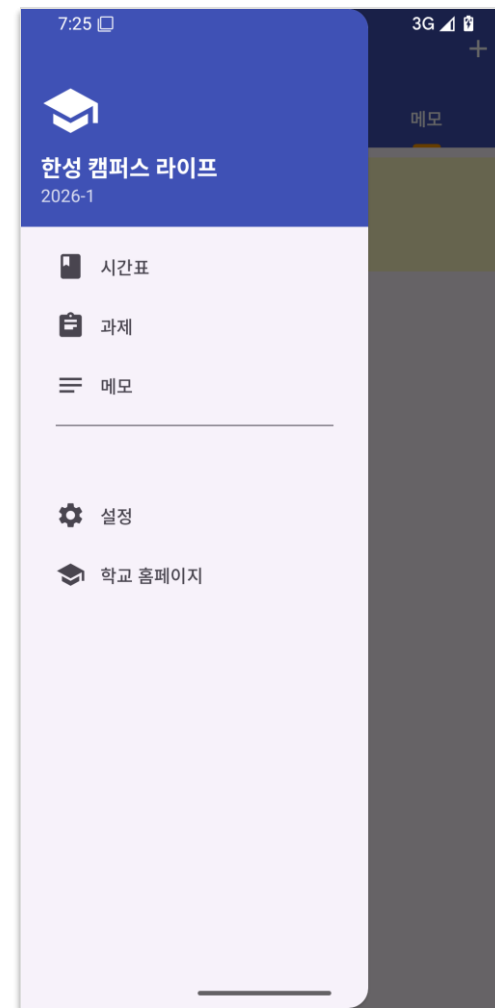
주제 선정, 배경

문제의식 - 앱이 세 개로 흩어져 있었다

- 시간표는 에브리타임, 과제 마감은 메모장, 잡다한 메모는 또 다른 앱
- 매번 앱을 옮겨 다니는 게 불편 -> 수업 보는 화면 하나에 다 모으자
- 마침 수업에서 Fragment, RecyclerView, SharedPreferences를 배워 직접 만들 수 있겠다 판단

목표

- 한 앱에서 시간표 + 과제 + 메모를 다 관리
- 외부 DB, 서버 없이 수업에서 배운 기술만으로 구현
- 실제로 내가 쓸 수 있을 만큼 동작이 매끄럽게



앱 구조와 기술 스택

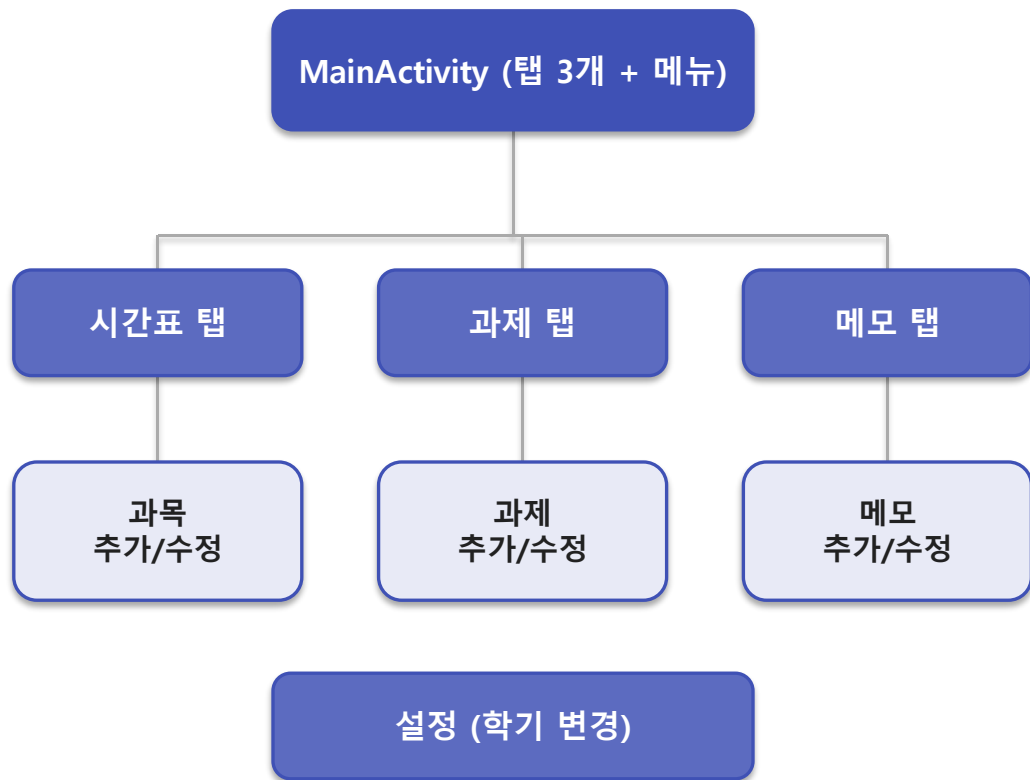
화면 구성

- MainActivity가 툴바, 드로어 메뉴, 뷰페이저를 들고 있다
- 탭 3개 - 시간표, 과제, 메모 (각각 Fragment)
- 각 탭에서 추가, 수정 화면(Activity)으로 이동
- 학기 변경은 별도 설정 화면

기술 스택

- 언어 - Kotlin, 화면 연결 - View Binding
- 화면 - Fragment, ViewPager2, RecyclerView, GridLayout
- 저장 - SharedPreferences (키-값)
- minSdk 24, targetSdk 36

화면 흐름



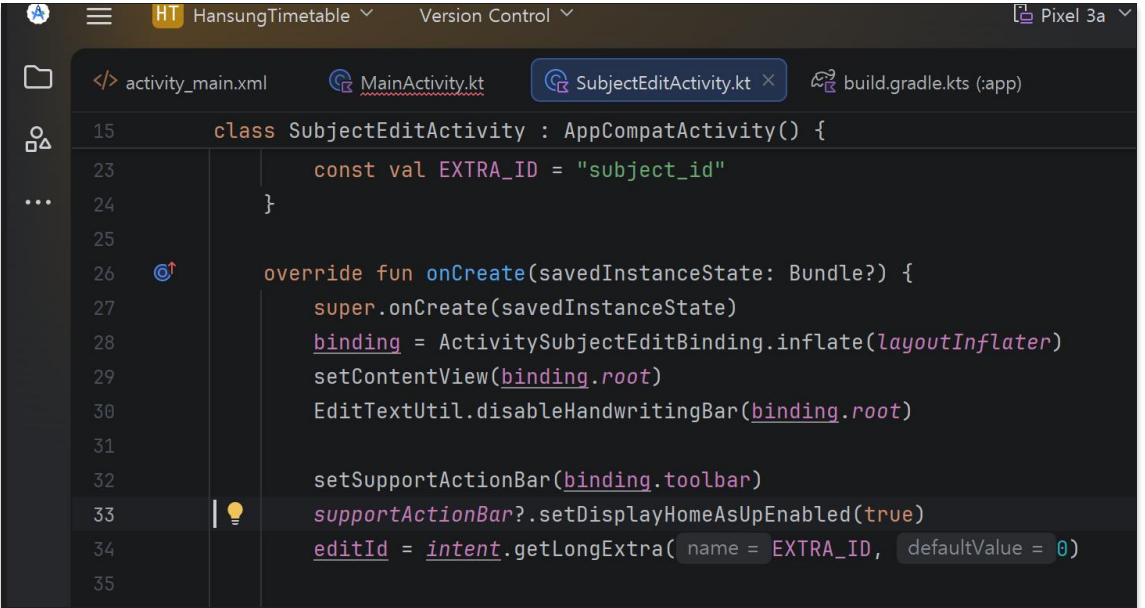
화면과 입력 - 뷰 바인딩과 위젯

뷰 바인딩 (View Binding)

- findViewById를 일일이 쓰지 않고 binding.뷰이름으로 바로 접근
- XML 파일명으로 바인딩 클래스가 자동으로 만들어진다
 - activity_main.xml -> ActivityMainBinding 처럼 변환
 - inflate()로 객체를 얻고 setContentView(binding.root)로 연결
- 덕분에 뷰를 잘못 찾는 실수가 줄고 코드가 짧아졌다

입력 위젯과 적용

- EditText로 강의명, 교수, 강의실, 메모를 입력받음
- 요일은 하나만 고르면 되니 RadioGroup으로 묶은 RadioButton
- 과제 완료 여부는 여러 개를 독립적으로 켜는 CheckBox



```
15 class SubjectEditActivity : AppCompatActivity() {
23     const val EXTRA_ID = "subject_id"
24 }
25
26 override fun onCreate(savedInstanceState: Bundle?) {
27     super.onCreate(savedInstanceState)
28     binding = ActivitySubjectEditBinding.inflate(layoutInflater)
29     setContentView(binding.root)
30     EditTextUtil.disableHandwritingBar(binding.root)
31
32     setSupportActionBar(binding.toolbar)
33     supportActionBar?.setDisplayHomeAsUpEnabled(true)
34     editId = intent.getLongExtra(name = EXTRA_ID, defaultValue = 0)
35 }
```

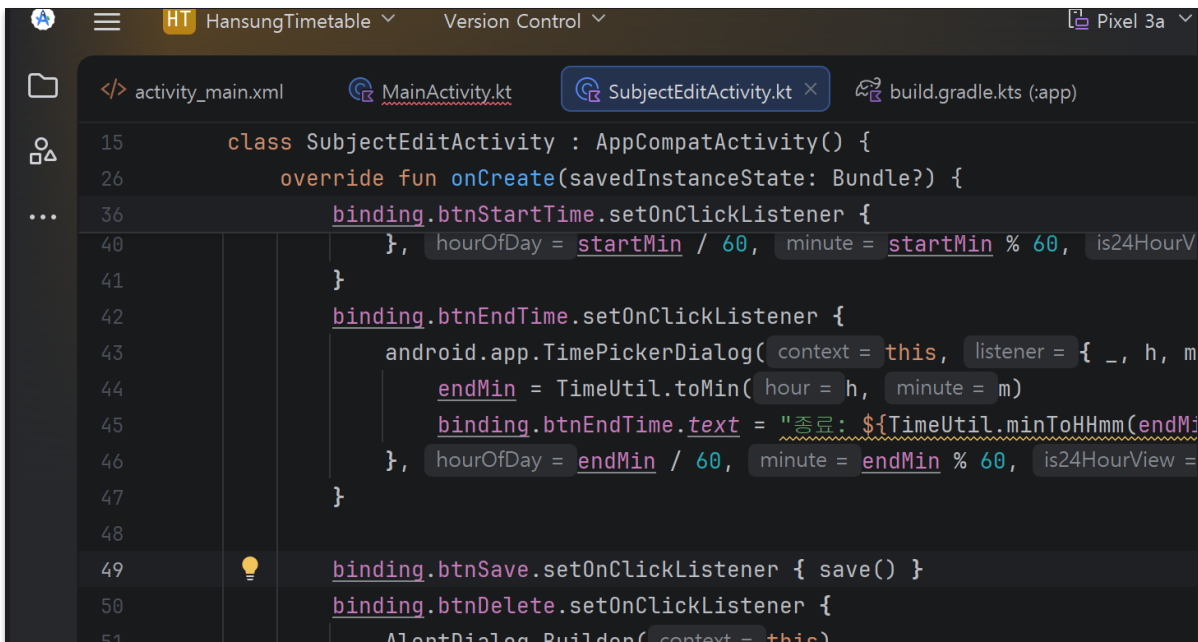
시간표를 그리는 레이아웃과 다이얼로그

레이아웃과 이벤트

- GridLayout - 표(행과 열) 형태로 뷰를 배치하는 레이아웃
 - rowCount, columnCount로 행과 열 개수를 정한다
 - 시간표가 격자라서 GridLayout이 가장 잘 맞았다
- 뷰 이벤트는 이벤트 소스 + 핸들러를 리스너로 연결
 - 셀을 setOnClickListener로 묶어 누르면 수정 화면으로 이동

다이얼로그

- Toast - 잠깐 떴다 사라지는 안내 메시지
- DatePicker, TimePicker - 날짜와 시간을 타이핑 없이 고르게
- AlertDialog - 삭제 전 확인, 과목 선택은 setSingleChoiceItems



```
15 class SubjectEditActivity : AppCompatActivity() {
26     override fun onCreate(savedInstanceState: Bundle?) {
36         binding.btnStartTime.setOnClickListener {
40             }, hourOfDay = startMin / 60, minute = startMin % 60, is24HourV
41         }
42         binding.btnEndTime.setOnClickListener {
43             android.app.TimePickerDialog( context = this, listener = { _, h, m
44                 endMin = TimeUtil.toMin( hour = h, minute = m)
45                 binding.btnEndTime.text = "종료: ${TimeUtil.minToHHmm(endM
46             }, hourOfDay = endMin / 60, minute = endMin % 60, is24HourView =
47         }
48
49         binding.btnSave.setOnClickListener { save() }
50         binding.btnDelete.setOnClickListener {
51             AlertDialog.Builder( context = this)
```

탭과 목록의 핵심 - 프래그먼트와 리사이클러뷰

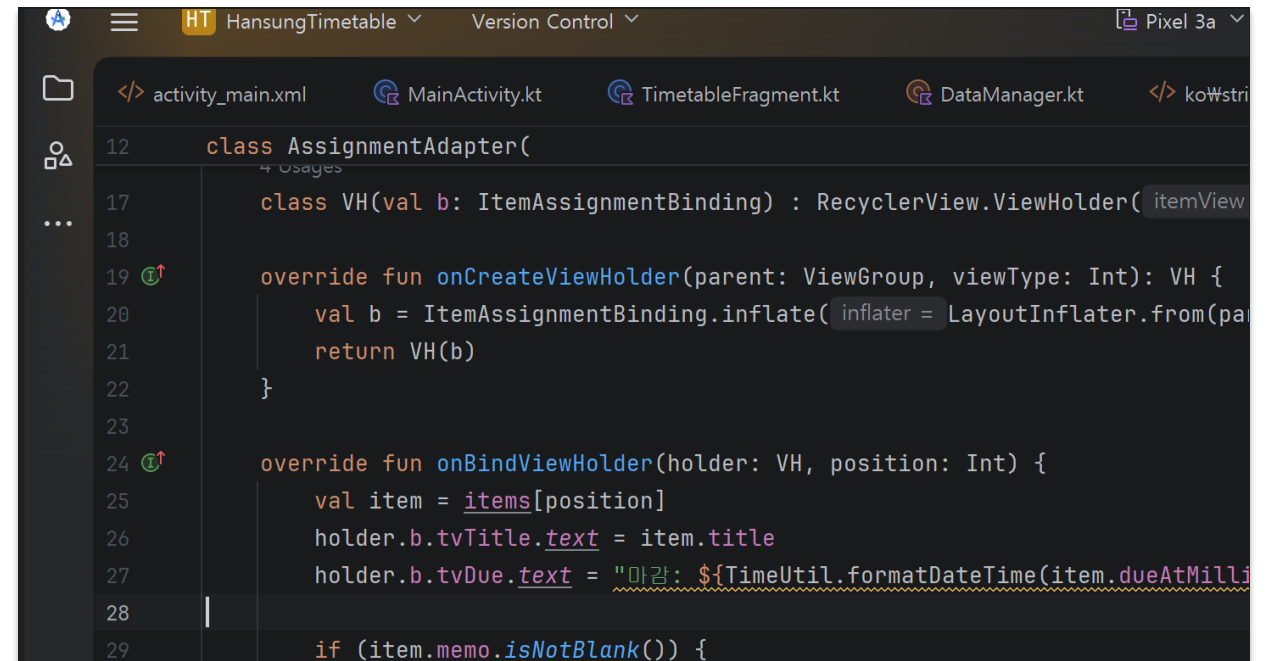
가장 많이 쓰는 부분

화면 구성

- Fragment - 액티비티처럼 동작하는 화면 조각, 탭마다 1개
- ViewPager2 - 좌우로 스와이프해 화면을 넘기는 뷰
 - 프래그먼트들은 FragmentStateAdapter로 연결
- DrawerLayout - 옆에서 서랍처럼 열리는 메뉴
- 툴바 메뉴는 onCreate / onPrepareOptionsMenu, 검색은 SearchView

목록 - RecyclerView

- ViewHolder(뷰 보관) + Adapter(항목 구성) + LayoutManager(배치)
- 화면에 보이는 만큼만 만들어 재활용해서 목록이 길어도 가볍다



```
12 class AssignmentAdapter(  
13     + Uses  
14 ) {  
15     class VH(val b: ItemAssignmentBinding) : RecyclerView.ViewHolder(b.rootView) {  
16     }  
17  
18     override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): VH {  
19         val b = ItemAssignmentBinding.inflate(LayoutInflater.from(parent.context), parent, false)  
20         return VH(b)  
21     }  
22  
23     override fun onBindViewHolder(holder: VH, position: Int) {  
24         val item = items[position]  
25         holder.b.tvTitle.text = item.title  
26         holder.b.tvDue.text = "마감: ${TimeUtil.formatDateTime(item.dueAtMilliSeconds)}"  
27  
28         if (item.memo.isNotBlank()) {  
29             holder.b.tvMemo.text = item.memo
```

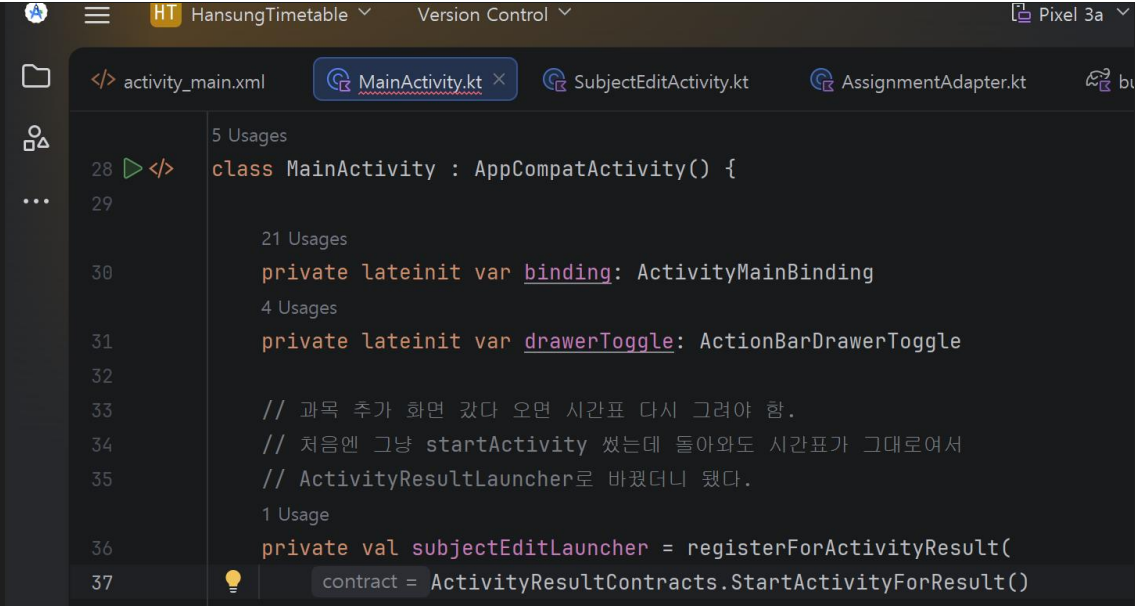
화면 이동과 새로고침 - 인텐트와 생명주기

인텐트 (Intent)

- 컴포넌트를 실행하려고 시스템에 전달하는 메시지
- 명시적 인텐트 - 실행할 클래스를 직접 지정
- 암시적 인텐트 - 동작만 알리면 시스템이 맞는 앱을 찾아줌
- ActivityResultLauncher - 다른 화면에 갔다가 결과를 받아옴

적용

- 추가, 수정 화면 이동은 명시적 인텐트
- 학교 홈페이지를 브라우저로 여는 건 암시적 인텐트
- 과목을 추가하고 돌아오면 결과를 받아 시간표를 다시 그림
- 화면이 다시 보일 때 onResume에서 목록을 새로고침



```
5 Usages
28 </> class MainActivity : AppCompatActivity() {
29
    21 Usages
30     private lateinit var binding: ActivityMainBinding
    4 Usages
31     private lateinit var drawerToggle: ActionBarDrawerToggle
32
33     // 과목 추가 화면 갔다 오면 시간표 다시 그려야 함.
34     // 처음엔 그냥 startActivity 썼는데 돌아와도 시간표가 그대로여서
35     // ActivityResultLauncher로 바꿨더니 됐다.
    1 Usage
36     private val subjectEditLauncher = registerForActivityResult(
37         contract = ActivityResultContracts.StartActivityForResult()
```

데이터 저장 - 공유된 프리퍼런스

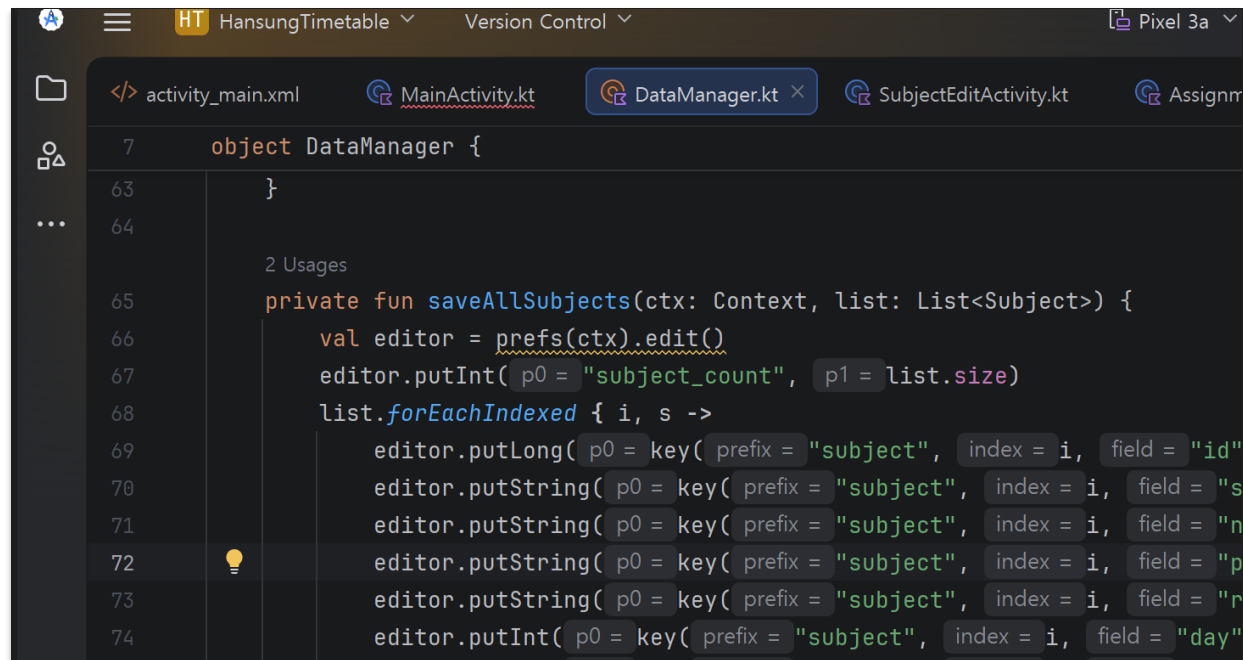
이 앱에서 가장 고민한 부분

공유된 프리퍼런스 (SharedPreferences)

- 데이터를 키-값 형태로 폰에 저장, 앱을 꺼도 남아있다
- Context.getSharedPreferences()로 앱 전체 데이터를 다룬다
 - Editor의 putInt, putString으로 저장하고 get으로 불러옴
- 학기 설정 화면은 PreferenceFragmentCompat로 만듦

목록을 저장하려고 직접 짰 구조

- 원래 설정값 저장용이라 목록을 넣으려면 직접 설계가 필요했다
- 개수는 count에, 항목은 prefix_번호_필드 규칙으로 펼쳐 저장
- 불러올 땐 count만큼 반복하며 다시 객체로 복원



```
object DataManager {  
    63 }  
    64  
    2 Usages  
    65 private fun saveAllSubjects(ctx: Context, list: List<Subject>) {  
    66     val editor = prefs(ctx).edit()  
    67     editor.putInt( p0 = "subject_count", p1 = list.size)  
    68     list.forEachIndexed { i, s ->  
    69         editor.putLong( p0 = key( prefix = "subject", index = i, field = "id"  
    70         editor.putString( p0 = key( prefix = "subject", index = i, field = "s  
    71         editor.putString( p0 = key( prefix = "subject", index = i, field = "n  
    72         editor.putString( p0 = key( prefix = "subject", index = i, field = "p  
    73         editor.putString( p0 = key( prefix = "subject", index = i, field = "r  
    74         editor.putInt( p0 = key( prefix = "subject", index = i, field = "day"
```

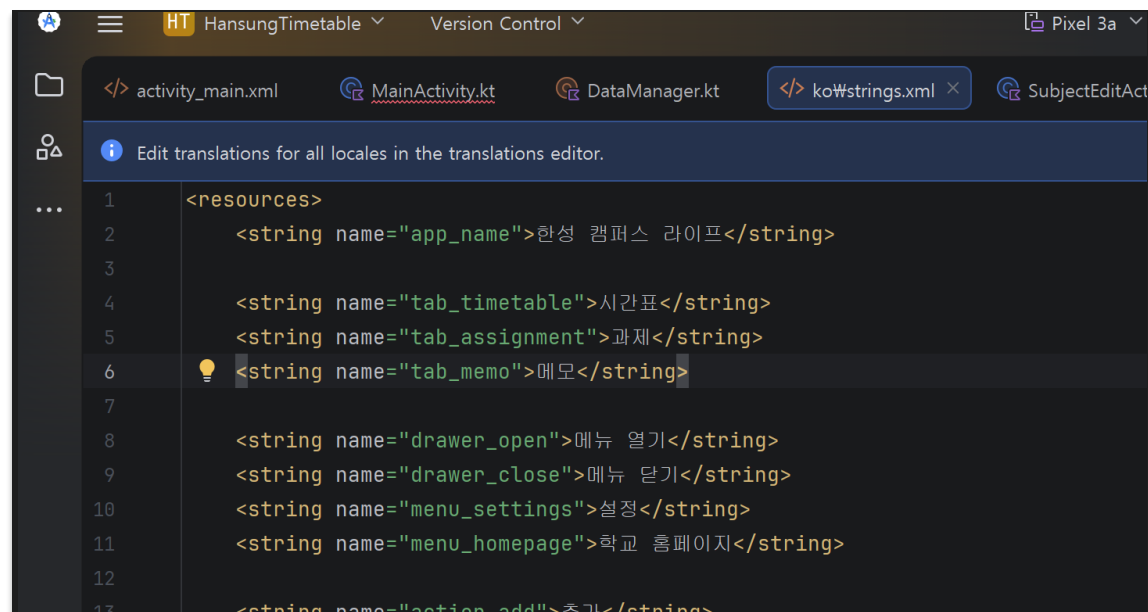

리소스 분리와 한국어 지원

리소스 분리

- 화면에 보이는 문구는 strings.xml, 색은 colors.xml로 따로 관리
 - 코드에 글자나 색을 직접 박지 않아 수정과 번역이 편하다
- 요일별 과목 색은 colors.xml의 파스텔 색을 골라 적용

다국어와 가로 화면

- values 폴더는 영어, values-ko 폴더는 한국어 문구
 - 폰 언어 설정에 맞춰 안드로이드가 자동으로 골라준다
- 시간표는 가로로 돌렸을 때를 위한 layout-land도 따로 만들



직접 찾아본 부분

수업에서 안 배운 것 - 공식 문서, 검색 참고

TabLayoutMediator

- 뷰페이지2 탭에 글자(시간표, 과제, 메모)를 붙여주는 연결

GridLayout에 코드로 칸 추가 (동적 addView)

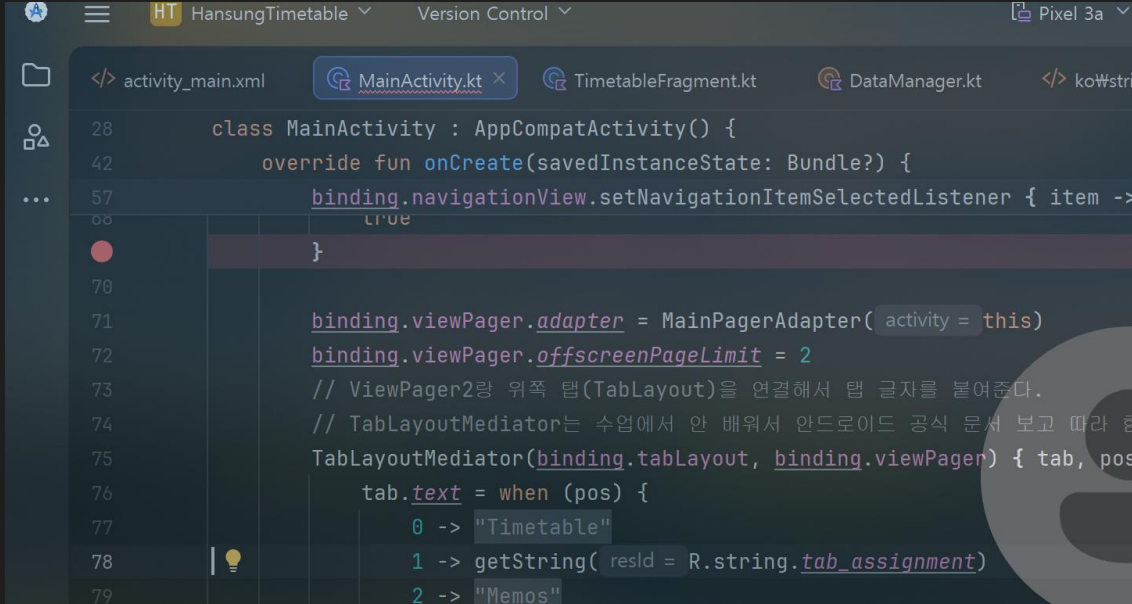
- 저장된 과목 수만큼 시간표 칸을 코드로 만들어 붙임

adjustResize

- 키보드가 올라올 때 입력창을 가리지 않게 화면을 밀어줌

privateImeOptions

- 입력창 옆에 뜨는 키보드 자체의 툴바를 숨김



```
28 class MainActivity : AppCompatActivity() {
42     override fun onCreate(savedInstanceState: Bundle?) {
57         binding.navigationView.setNavigationItemSelectedListener { item ->
68             true
69         }
70
71         binding.viewPager.adapter = MainPagerAdapter( activity = this)
72         binding.viewPager.offscreenPageLimit = 2
73         // ViewPager2랑 위쪽 탭(TabLayout)을 연결해서 탭 글자를 붙여준다.
74         // TabLayoutMediator는 수업에서 안 배워서 안드로이드 공식 문서 보고 따라 함
75         TabLayoutMediator(binding.tabLayout, binding.viewPager) { tab, pos
76             tab.text = when (pos) {
77                 0 -> "Timetable"
78                 1 -> getString( resId = R.string.tab_assignment)
79                 2 -> "Memos"
```

문제점과 해결

실제로 겪은 버그 (10점)

1) 과목을 추가했는데 시간표에 안 났다

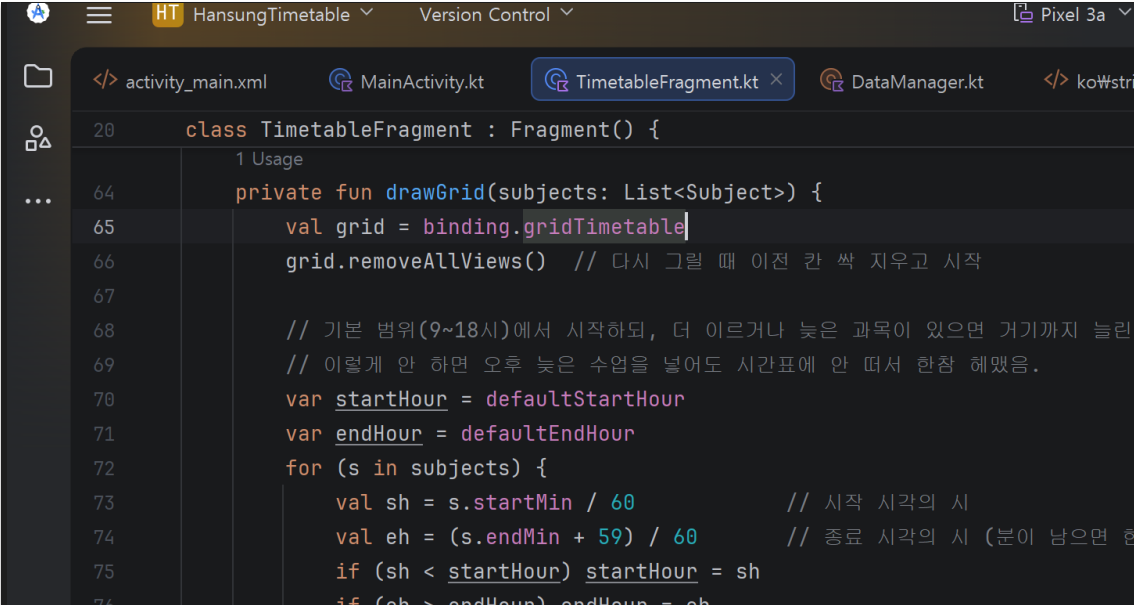
- 원인 - 추가 화면에서 돌아와도 Fragment가 다시 안 그려짐
- 해결 - 수업에서 배운 ActivityResultLauncher로 결과를 받아 갱신

2) 시간표 범위 밖(이르거나 늦은) 과목이 안 보였다

- 원인 - 표시 범위가 9시-18시로 고정돼 있었음
- 해결 - 과목 시간에 맞춰 표시 범위를 자동으로 넓힘

3) 키보드가 입력창을 가렸다

- 해결 - Manifest에 windowSoftInputMode=adjustResize 추가



```
20 class TimetableFragment : Fragment() {
    1 Usage
    64 private fun drawGrid(subjects: List<Subject>) {
    65     val grid = binding.gridTimetable
    66     grid.removeAllViews() // 다시 그릴 때 이전 칸 싹 지우고 시작
    67
    68     // 기본 범위(9~18시)에서 시작하되, 더 이르거나 늦은 과목이 있으면 거기까지 늘림
    69     // 이렇게 안 하면 오후 늦은 수업을 넣어도 시간표에 안 떠서 한참 헤맸음.
    70     var startHour = defaultStartHour
    71     var endHour = defaultEndHour
    72     for (s in subjects) {
    73         val sh = s.startMin / 60 // 시작 시각의 시
    74         val eh = (s.endMin + 59) / 60 // 종료 시각의 시 (분이 남으면 한
    75         if (sh < startHour) startHour = sh
    76         if (eh > endHour) endHour = eh
    }
```

직접 더한 기능 - 과제 D-day

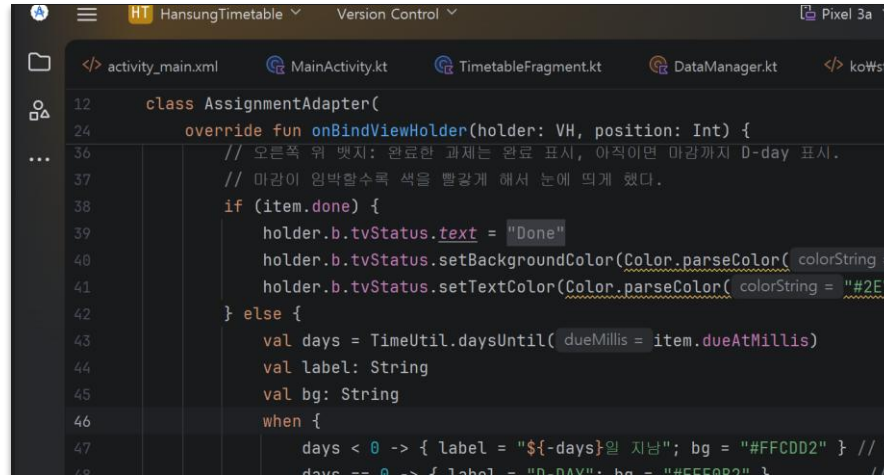
구현 디벨롭

왜 넣었나

- 과제는 마감이 언제인지가 제일 중요한데
- 날짜만 적혀 있으면 잘 안 와닿았다
- 며칠 남았는지 D-day 배지로 한눈에 보이게 직접 추가

어떻게

- TimeUtil에 날짜 계산 함수(daysUntil)를 직접 작성
 - 오늘과 마감일을 0시로 맞춰 날짜 차이만 계산
- 남은 일수에 따라 색을 구분
 - 지남은 빨강, 임박은 주황, 여유는 파랑
- 완료한 과제는 초록색 완료 배지



```
12 class AssignmentAdapter(  
24     override fun onBindViewHolder(holder: VH, position: Int) {  
36         // 오른쪽 위 배지: 완료한 과제는 완료 표시, 아직이면 마감까지 D-day 표시.  
37         // 마감이 임박할수록 색을 빨강계 해서 눈에 띄게 했다.  
38         if (item.done) {  
39             holder.b.tvStatus.text = "Done"  
40             holder.b.tvStatus.setBackgroundColor(Color.parseColor( colorString =  
41                 holder.b.tvStatus.setTextColor(Color.parseColor( colorString = "#2E7D32"  
42         } else {  
43             val days = TimeUtil.daysUntil( dueMillis = item.dueAtMillis)  
44             val label: String  
45             val bg: String  
46             when {  
47                 days < 0 -> { label = "${-days}일 지남"; bg = "#FFCDD2" } // 빨  
48                 days == 0 -> { label = "D-DAY"; bg = "#FFB74D" } // 주  
49                 days > 0 -> { label = "${days}일 여유"; bg = "#BBDEFB" } // 파  
50         }
```



결과 분석과 개선 아이디어

결과, 한계, 다음 단계 (10점)

결과 - 잘 된 것

- 시간표, 과제, 메모에 추가/수정/삭제/검색/D-day/학기설정까지 동작
- 빌드 성공, 에뮬레이터에서 크래시 없이 정상 실행 확인

한계 - 솔직하게

- 공유된 프리퍼런스는 폰 한 대에만 저장되고, 데이터가 많아지면 비효율적
- 과제 마감 알림이나 기기 간 동기화가 아직 없음

개선 아이디어 - 이후 수업에서 배운 기술을 적용한다면

- 파이어베이스 Firestore - 폰에서 클라우드로 바꿔 여러 기기에서 동기화
- FCM 푸시 알림 - 과제 마감 전날 알림 / 인증 - 로그인으로 개인별 시간표
- 구글 지도 - 강의실 위치 표시 / Compose - 화면 UI 코드 간결화

시연 및 마무리

1분 시연

1분 시연 순서

- 앱을 실행하고 탭 3개를 좌우로 스와이프 (시간표, 과제, 메모)
- 플러스로 과목을 추가하면 시간표에 바로 뜨는 것 확인
- 과제 탭에서 추가하면 D-day가 같이 뜨고, 위에서 검색
- 메모 탭에서 메모 추가
- 드로어 메뉴에서 학기 변경, 학교 홈페이지 열기

수업에서 배운 걸 한 앱에 모아보면서,
화면이 어떻게 연결되고 데이터가 어떻게 저장되는지
직접 부딪혀 가며 익혔습니다. 감사합니다.

